



SAPIENZA
UNIVERSITÀ DI ROMA

Sapienza-DC: progettazione e sviluppo di un sistema RAG domain-aware

Implementazione di un chatbot multi-dominio basato su Retrieval-Augmented Generation

Facoltà di Ingegneria dell'Informazione, Informatica e Statistica (I3S)
Dipartimento di Ingegneria Informatica Automatica e Gestionale (DIAG)
Corso di Laurea in Ingegneria Informatica e Automatica

Fabio Pastore

Matricola 2129632

Relatore

Prof. Roberto Navigli

Anno Accademico 2025/2026

Sapienza-DC: progettazione e sviluppo di un sistema RAG domain-aware

Tesi di laurea triennale. Sapienza Università di Roma

© 2026 Fabio Pastore. Tutti i diritti riservati

Questa tesi è stata composta con $\text{\LaTeX}$ e la classe Sapthesis.

Email dell'autore: pastore.2129632@studenti.uniroma1.it

To my family and friends.

Sommario

Nella seguenti tesi viene descritta la realizzazione di Sapienza-DC, un chatbot RAG multi-dominio capace di rispondere alle richieste degli utenti mediante analisi e validazione di informazioni estratte da pagine web, fornendo risposte coerenti affiancate da un punteggio di affidabilità nonché dalle fonti utilizzate dall’LLM per rispondere.

I capitoli contenuti in questo elaborato tratteranno gli obiettivi fondamentali di Sapienza-DC, la raccolta dei requisiti di progetto, l’analisi delle tecnologie utilizzate, l’architettura del sistema, la progettazione, la fase di implementazione e gli sviluppi del progetto, proposti in ordine cronologico; la gestione del contesto dell’LLM e un excursus sulle ottimizzazioni introdotte; la sicurezza della pipeline dinanzi a tentativi di prompt injection e una valutazione qualitativa, sotto forma di benchmark, del sistema complessivo.

Indice

1	Introduzione	1
1.1	Presentazione del progetto	1
1.2	Requisiti di progetto	1
2	Analisi delle tecnologie impiegate	3
2.1	Concetti preliminari di NLP	3
2.1.1	Large Language Model (LLM)	4
2.1.2	Retrieval-Augmented Generation (RAG)	4
2.1.3	Bi-encoder, cross-encoder	5
2.2	Framework, strumenti e librerie di sviluppo	6
2.2.1	FastAPI	6
2.2.2	LangChain	6
2.2.3	Hugging Face	7
2.2.3.1	FastEmbed	7
2.2.3.2	SentenceTransformers	7
2.2.4	Ollama e llama.cpp	7
2.2.5	SearXNG	9
2.2.6	Crawl4AI	9
2.2.7	Jinja2	9
2.2.8	TailwindCSS	9
2.3	Containerizzazione ed orchestrazione	9
2.3.1	Architettura a microservizi: Docker e Docker Compose	9
2.4	Database relazionale	9
2.4.1	MariaDB	9
3	Progettazione dell'applicazione	11
4	Architettura del sistema	13
5	Implementazione della pipeline RAG	15
6	Gestione del contesto e generazione delle risposte	17
7	Ottimizzazioni, sicurezza e benchmark	19
8	Conclusione	21

Capitolo 1

Introduzione

1.1 Presentazione del progetto

Sapienza-DC è stato realizzato come progetto finale per il corso di Laboratorio di Ingegneria Informatica.

In seguito alla realizzazione di un progetto preliminare, il quale consisteva nella realizzazione di un parser per opportuni domini assegnati a ciascun gruppo partecipante, sono stati selezionati i migliori progetti, al fine di avere l'opportunità di accedere ad un progetto personalizzato.

Tra i vari progetti personalizzati veniva proposta la realizzazione di un chatbot di dominio capace di ricevere in input una query da parte dell'utente, selezionare un dominio su cui cercare informazioni per rispondere alla domanda, ricercare fonti su pagine web appartenenti a quest'ultimo, estrarre e processare i contenuti rilevanti da fornire a un LLM avente il compito di rispondere al quesito posto, fornendo aggiuntivamente le fonti di riferimento impiegate.

1.2 Requisiti di progetto

Le specifiche fornite preliminarmente per la realizzazione del progetto consistevano in una descrizione generale dell'architettura della pipeline da implementare:

- consentire all'utente di porre al proprio sistema una query;
- assicurarsi che la query dell'utente possa essere risolta sfruttando i domini disponibili;
- ricercare online pagine (nei domini disponibili) che possano contenere informazioni utili;
- parsare le pagine (sfruttando i parser dell'esonero) e fornire ad un LLM la query assieme ai dati estratti da queste ultime;
- restituire la risposta all'utente.

In aggiunta a quanto appena elencato, si richiedevano inoltre diverse funzionalità aggiuntive. In particolare, il sistema implementato sarebbe dovuto essere capace di: gestire richieste ambigue, chiedendo, ove necessario, chiarimenti all'utente; rigettare la query dell'utente qualora quest'ultima fosse ritenuta inaccettabile o inopportuna; classificare i contenuti delle pagine web per grado di pertinenza alla domanda, valutando eventualmente la possibilità di estrarre dati da domini qualunque in caso di query esterna ai domini assegnati; riassumere o selezionare porzioni delle informazioni ottenute in fase di parsing per evitare di saturare la context window del large language model; citare le fonti e stimare l'affidabilità della risposta fornita, gestendo opportunamente il caso in cui le informazioni in possesso fossero parziali o insufficienti per rispondere alla domanda dell'utente.

Di seguito viene proposto il diagramma utilizzato per presentare i requisiti di progetto.

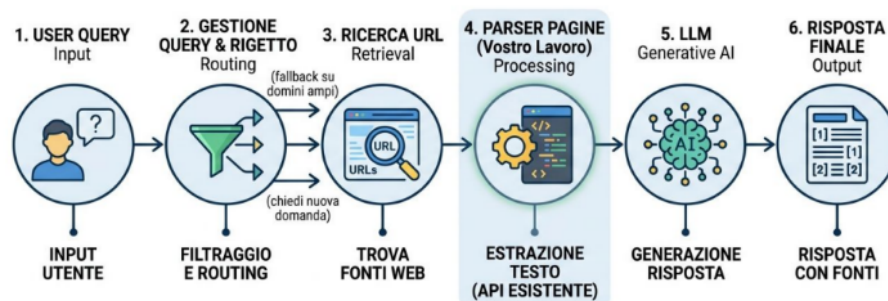


Figura 1.1. Diagramma esemplificativo fornito assieme ai requisiti di progetto.

Si noti come tali specifiche rappresentino solo una linea guida alla realizzazione del progetto, descrivendo solo gli aspetti generali e le funzionalità fondamentali che quest'ultimo dovrebbe implementare.

Questo è voluto: tra gli obiettivi del progetto personalizzato vi è proprio quello di lasciare al gruppo la libertà, e dunque anche le difficoltà che ne conseguono, di produrre quanto richiesto utilizzando gli strumenti nonché tecnologie ritenute più consone.

A tal riguardo, nel capitolo che segue introdurremo le principali tecnologie e framework utilizzati per sviluppare Sapienza-DC.

Capitolo 2

Analisi delle tecnologie impiegate

In questo capitolo verranno fornite informazioni inerenti alle principali tecnologie utilizzate durante lo sviluppo di Sapienza-DC, in modo tale da chiarire il ruolo e il funzionamento di ciascuno strumento impiegato durante la fase di implementazione del progetto.

2.1 Concetti preliminari di NLP

L’NLP (Natural Language Processing) è una branca dell’Intelligenza Artificiale che studia la lettura, comprensione, interpretazione nonché generazione, da parte di un calcolatore, di linguaggio umano, in modo naturale.

NLP combina in un unicum la linguistica computazionale, ossia le regole del linguaggio, con tecniche avanzate di machine learning¹ e deep learning². In una tipica pipeline NLP, il testo viene dapprima elaborato mediante tokenizzazione, la quale consiste nella suddivisione del testo in unità più piccole, quali parole o frasi. Dopodiché, ciascun token viene standardizzato, mediante conversione in minuscolo di ciascun carattere, la rimozione delle cosiddette stop word (parole del linguaggio comune utilizzate frequentemente, come "è", "la", etc.) nonché caratteri speciali o di punteggiatura, considerati come rumore in questa fase iniziale.

Successivamente, ciascun token viene trasformato, mediante opportune tecniche di NLP, in rappresentazioni numeriche su cui un calcolatore può operare. Ad esempio, tramite opportuni modelli di embedding (incorporamento) è possibile trasformare un token in un vettore n-dimensionale di reali, ove è possibile memorizzare il significato semantico del token originale. Questo, peraltro, sarà proprio l’approccio utilizzato da Sapienza-DC, come vedremo più in avanti.

¹ machine learning: branca dell’IA avente come obiettivo quello di apprendere autonomamente da un insieme di dati in modo tale da essere in grado successivamente di effettuare previsioni su nuovi dati.

² deep learning: sottoinsieme del machine learning in cui si utilizzano reti neurali multistrato costituite da neuroni artificiali interconnessi.

A partire dai dati così ottenuti, vengono estratte ulteriori informazioni mediante opportune tecniche computazionali, al fine di identificare ruoli grammaticali delle parole, argomenti e sentiment, vale a dire il tono emotivo del testo. Infine, le nozioni raccolte vengono utilizzate per addestrare modelli di machine learning, capaci di apprendere tali relazioni e utilizzarle una volta completato l'addestramento per generare linguaggio naturale.

2.1.1 Large Language Model (LLM)

Un LLM è un particolare tipo di modello linguistico capace di generare linguaggio naturale mediante addestramento su una grande quantità di dati. Il termine "large" deriva dal grande (spesso nell'ordine di miliardi) numero di parametri del modello probabilistico appresi in fase di training. Gli LLM sono basati su deep learning e utilizzano come componenti fondamentali reti neurali e trasformatori³ (transformers).

Un LLM genera testo in modo puramente statistico, infatti, se $(w_1, w_2, \dots, w_T)$ sono parole che costituiscono una frase f , allora la probabilità che l'LLM $\mathcal{L}$ generi f è data da:

$$\mathbb{P}(w_1, w_2, \dots, w_T) = \prod_{j=1}^T \mathbb{P}(w_j | w_1, w_2, \dots, w_{j-1})$$

dove la probabilità di generare la j -esima parola w_j dipende dai parametri appresi durante l'addestramento del modello. La generazione, detta autoregressiva, avviene un token alla volta ed è pertanto sequenziale: ciascun nuovo token dipenderà direttamente da quelli generati in precedenza. Vi sarebbero numerosissime tematiche da approfondire a tal riguardo, ma, per brevità, considereremo sufficienti queste nozioni intuitive.

2.1.2 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation, ossia la generazione potenziata mediante recupero di dati, è un approccio che consiste nell'ottimizzazione dell'output di un LLM.

RAG prevede l'estrazione di informazioni e conoscenze utili da fonti autorevoli, al fine di accrescere le nozioni possedute dal modello. Tale strategia diviene estremamente utile quando è necessario che l'LLM fornisca una risposta a query altamente specifiche o inerenti a fatti recenti, altrimenti non contenuti nel dataset di addestramento dell'LLM.

Questa tecnica è adottata da Sapienza-DC per estrarre contenuti ed informazioni domain-specific con lo scopo di generare una risposta alle query proposte al chatbot da parte dell'utente.

³ trasformatore: particolare architettura di rete neurale artificiale che permette l'elaborazione parallela di diverse porzioni di testo, cogliendo inoltre eventuali relazioni tra parole distanti nel testo grazie a un meccanismo noto come self-attention.

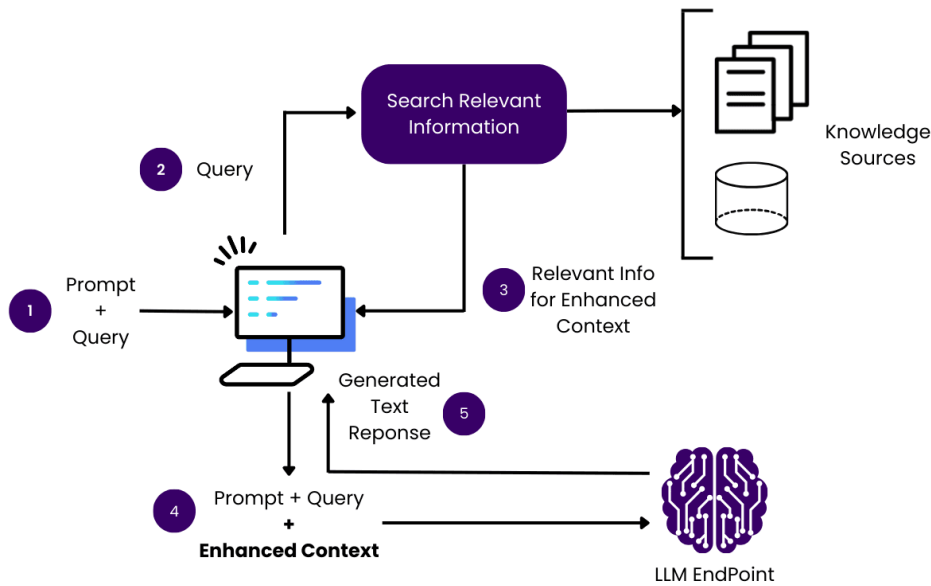


Figura 2.1. Elementi fondamentali di uno schema di RAG.⁴

2.1.3 Bi-encoder, cross-encoder

Un bi-encoder (o dual-encoder) è un modello utilizzato per convertire testo in vettori di reali, detti embedding dell'informazione di partenza. Il nome deriva dall'utilizzo di due reti neurali separate aventi lo scopo di trasformare rispettivamente query e documenti di riferimento nei relativi embedding.

L'idea a questo punto è molto semplice: verificare, mediante opportune operazioni sui vettori così ottenuti, quali documenti, o porzioni di essi, risultino più semanticamente affini alla query fornita dallo user. Un approccio utile a questo scopo che vedremo successivamente consisterà nel calcolo della cosine similarity tra v_q e v_c , vettori rispettivamente della query q e del chunk c , appartenente a un determinato documento ottenuto dalla corrispettiva pagina web.

I bi-encoder sono notoriamente veloci, tuttavia, analizzando separatamente query e chunk dei documenti non godono di un'accuratezza eccellente. Per questo motivo è necessario utilizzare anche i cross-encoder. Questi ultimi sono modelli aventi lo stesso identico scopo, ma che offrono una precisione molto maggiore dei dual-encoder, a discapito di un maggiore costo computazionale.

In particolare, i cross-encoder devono necessariamente elaborare contemporaneamente query e chunk di testo, sfruttando i meccanismi di attenzione offerti dai transformer affinché sia colto il contesto della domanda e la reale pertinenza tra quest'ultima e la porzione di documento sotto esame. L'output di un cross-encoder non consiste in un vettore, ma in uno score, indice del grado di rilevanza del chunk c rispetto alla query q .

⁴ Fonte immagine: obot.ai. Rilasciata sotto la seguente licenza: CC BY 4.0.

Nella fase di implementazione, utilizzeremo i cross-encoder, in seguito a un filtraggio preliminare mediante bi-encoder e cosine similarity, per effettuare reranking (riordinamento) dei chunk vari chunk estratti dalle fonti di interesse.

2.2 Framework, strumenti e librerie di sviluppo

2.2.1 FastAPI

FastAPI è un framework web moderno, open-source e ad alte prestazioni, progettato per la sviluppo di API⁵ (Application Programming Interface) REST (REpresentational State Transfer) in Python.

Rilasciato nel 2018, FastAPI è ben presto divenuto uno degli standard per lo sviluppo backend in Python, grazie alla sua efficienza nonché facilità di utilizzo.

Il framework usufruisce di due librerie fondamentali quali Starlette, un toolkit web ASGI (Asynchronous Server Gateway Interface) per la gestione di servizi asincroni, e Pydantic, grazie a cui è possibile una validazione automatica e personalizzabile dei dati e la creazione di modelli Pydantic per la definizione dei tipi di input e output di ciascun endpoint⁶.

Tali funzionalità facilitano e rendono estremamente intuitivo lo sviluppo di backend, garantendo prestazioni equiparabili ad altre tecnologie popolari come Node.js per lo sviluppo server-side.

Tra le funzionalità più apprezzate di FastAPI risalta la generazione automatica della documentazione API. Ogni qualvolta viene definito un nuovo endpoint, FastAPI genera automaticamente documentazione e interfacce Web UI (Swagger UI) mediante cui inviare richieste ai fini, ad esempio, di testing dell'applicazione.

Infine, il framework segue e adotta i principali standard web moderni per lo sviluppo di API basate su HTTP (HyperText Transfer Protocol), quali OpenAPI e JSON Schema per la validazione nonché serializzazione dei dati scambiati.

2.2.2 LangChain

LangChain è un framework open-source progettato per facilitare lo sviluppo di applicazioni che fanno uso di LLM.

Gli impieghi possibili di LangChain sono numerosi, ma ai fini dello sviluppo di Sapienza-DC ci limiteremo a utilizzare la classe RecursiveCharacterTextSplitter per

⁵ API - Application Programming Interface: un insieme di regole e protocolli che permettono a due applicazioni di comunicare correttamente.

⁶ endpoint: nell'ambito delle API, un URL specifico tramite cui un'applicazione riceve ed elabora richieste API

l'estrazione di chunk coerenti dalle fonti di riferimento, preservando la semantica delle informazioni originali.

2.2.3 Hugging Face

Hugging Face è una piattaforma open-source dedicata all'intelligenza artificiale e al machine learning. Grazie alla libreria Model Hub è possibile il download di modelli già addestrati per svariati scopi, tra cui quelli di cui usufruiremo ai fini dello sviluppo del progetto.

2.2.3.1 FastEmbed

FastEmbed è una libreria Python adatta a generare embedding di testi e immagini. Come già accennato in precedenza l'embedding permette di ottenere un vettore di reali che mantengano la semantica linguistica, nel caso di testo, del dato originale.

Tra gli innumerevoli modelli offerti da FastEmbed si è scelto di utilizzare `intfloat/multilingual-e5-small` come bi-encoder, il cui download è possibile grazie alla Model Hub di Hugging Face.

2.2.3.2 SentenceTransformers

SentenceTransformers è un framework ideato per la computazione di embedding e l'utilizzo di modelli di reranking. Utilizzeremo quest'ultimo per poter disporre di `ms-marco-MiniLM-L-6-v2`, un cross-encoder adatto ai nostri scopi. Anche in questo caso il download del modello avverrà automaticamente tramite Hugging Face durante l'inizializzazione della nostra applicazione.

2.2.4 Ollama e llama.cpp

Ollama è un framework open-source concepito per semplificare l'esecuzione, la configurazione e la gestione locale degli LLM su sistemi operativi come macOS, Windows e Linux.

Il framework espone un set di API RESTful standard che consente di interagire programmaticamente con i modelli scaricati. Grazie al supporto di librerie ufficiali e sviluppate dalla community per linguaggi come Python e JavaScript, è possibile integrare le capacità di ragionamento degli LLM all'interno di applicazioni, script o flussi di lavoro aziendali in modo estremamente semplice.

Ollama offre, oltretutto, un'immagine Docker ufficiale da eseguire in un opportuno container. Maggiori informazioni su Docker e sul concetto di container verranno fornite nella relativa sezione di questo capitolo.

Si noti per di più che, poiché i modelli sono eseguiti interamente in locale, il sistema non trasmette alcuna informazione a server esterni, garantendo massima privacy e totale indipendenza da connessioni internet o provider terzi.

Inoltre, giacché l'elaborazione grava esclusivamente sull'hardware dell'utente, il costo marginale per ogni singola query posta all'LLM diviene fondamentalmente nullo.

In aggiunta a ciò, il framework supporta e aggiorna costantemente una libreria di modelli open weights, tra cui Llama, Mistral, Gemma e Phi-3, permettendo di selezionare l'architettura più idonea in base al caso d'uso specifico.

Per permettere a reti neurali complesse di funzionare su hardware consumer, Ollama implementa un processo noto come quantizzazione dei modelli. Questa tecnica riduce la precisione matematica dei pesi dell'LLM, abbattendo pertanto in maniera drastica la quantità di RAM nonché la potenza di calcolo richieste per l'esecuzione del modello.

Tuttavia, questo approccio comporta alcuni trade-off inevitabili. In primis, a causa dei limiti hardware dei dispositivi consumer, i modelli eseguiti localmente presentano un numero di parametri inferiore e, conseguentemente, una qualità dell'output e capacità di ragionamento più limitate.

In secondo luogo, l'effettiva velocità di esecuzione di Ollama è rigidamente vincolata all'hardware della macchina ospitante, prediligendo architetture basate su GPU a quelle CPU. Naturalmente, l'esecuzione di un LLM tramite Ollama è comunque possibile su macchine sprovviste di GPU, a patto di accettare un'inferenza più lenta del modello.

L'obiettivo di Sapienza-DC consisterà difatti nel permettere a qualsiasi utente, dotato di una scheda video o anche semplicemente del proprio processore, di eseguire il progetto localmente, fornendo la possibilità di affidarsi, in caso di necessità, anche ad API esterne, qualora l'hardware della propria macchina dovesse risultare non sufficientemente performante. Entreremo maggiormente nei dettagli di questa tematica nei capitoli successivi.

A tal proposito, vedremo come sia possibile utilizzare direttamente llama.cpp per gestire manualmente il caricamento, la gestione nonché l'esecuzione di query verso LLM locali, con lo scopo di ridurre l'overhead introdotto da Ollama, il quale non è altro che un wrapper di quest'ultimo.

Brevemente, llama.cpp è un'implementazione in linguaggio C/C++ del codice di inferenza dei modelli LLaMA, successivamente esteso a molteplici altre architetture, il cui scopo fondamentale consiste nel massimizzare la velocità di esecuzione locale di LLM. Rimuovendo l'overhead dovuto a Ollama e accettando la maggiore difficoltà implementativa che consegue da questa scelta, saremo in grado di minimizzare i requisiti hardware richiesti per eseguire modelli localmente, soprattutto su macchine sprovviste di GPU.

2.2.5 SearXNG

2.2.6 Crawl4AI

2.2.7 Jinja2

2.2.8 TailwindCSS

2.3 Containerizzazione ed orchestrazione

2.3.1 Architettura a microservizi: Docker e Docker Compose

2.4 Database relazionale

2.4.1 MariaDB

Capitolo 3

Progettazione dell'applicazione

Capitolo 4

Architettura del sistema

Your content goes here. If you compile this file, it will run perfectly without any errors, and the official logo will be placed precisely where the University guidelines require.

Capitolo 5

Implementazione della pipeline RAG

Fase implementativa.

Capitolo 6

Gestione del contesto e generazione delle risposte

Capitolo 7

Ottimizzazioni, sicurezza e benchmark

Capitolo 8

Conclusione

